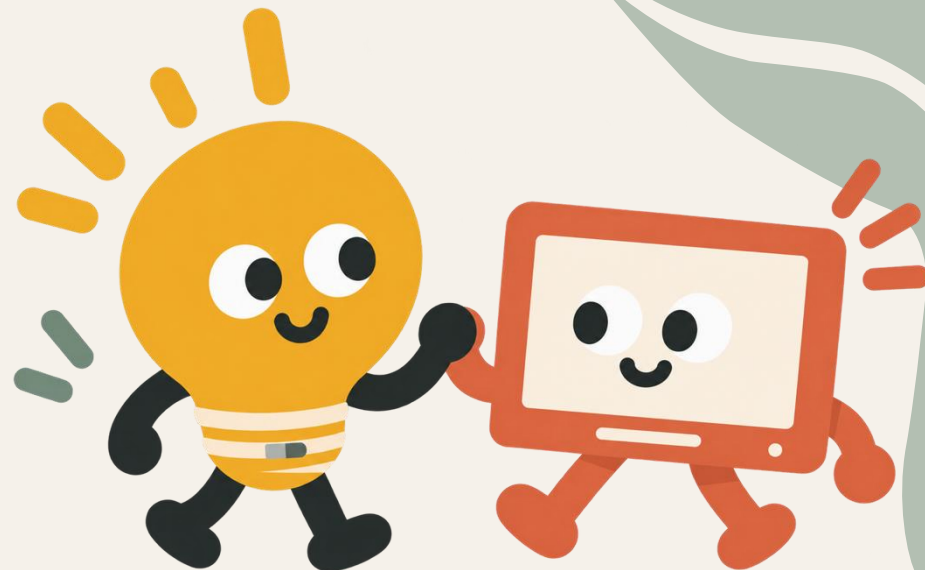


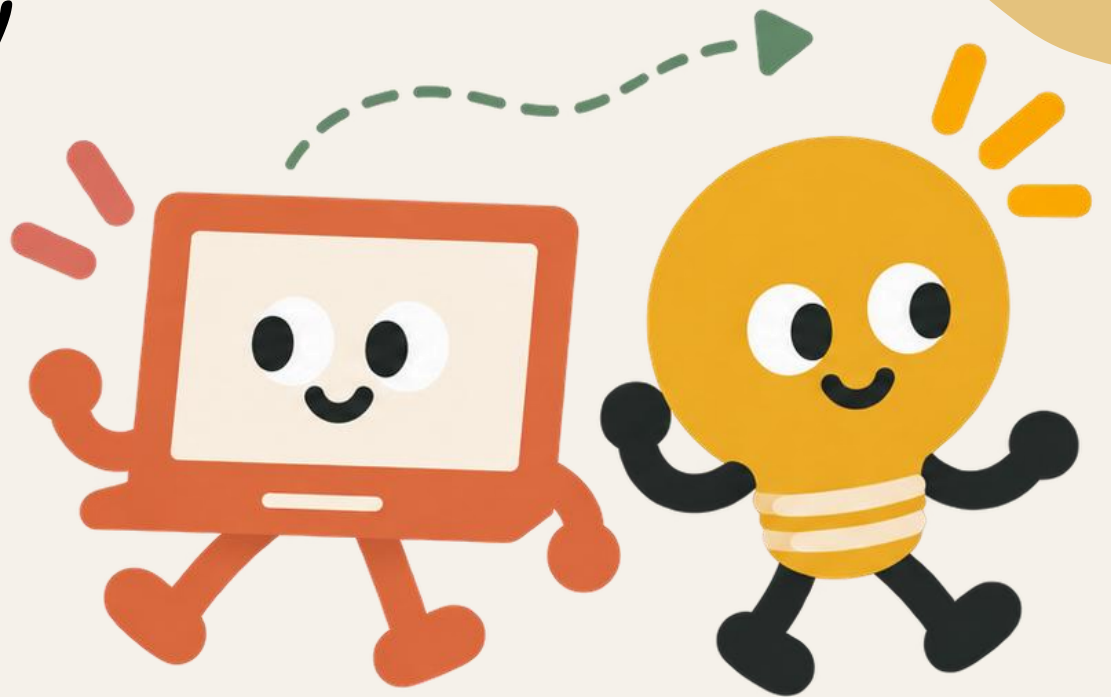
Taller teórico-práctico

Del dataset a U-Net: segmentación de imágenes con Deep Learning

→ Entorno y reproducibilidad



Directorio de trabajo y tipos de rutas



¿Dónde guardamos todo?



La estructura del proyecto también forma parte de la reproducibilidad.



.py vs .ipynb



 script.py

```
1 import numpy as np
2 from utils import procesar_imagen
3
4 def main():
5     img = cargar_imagen("gato.jpg")
6     mask = segmentar(img)
7     guardar_resultados(mask)
8
9 if __name__ == "__main__":
10     main()
```

 Programa Python .py

-  Código organizado como archivo de texto
-  Normalmente se ejecuta de principio a fin
-  Cada nueva ejecución comienza de nuevo
-  Orientado a programas y código reutilizable
-  Salidas normalmente en terminal/archivos
-  Muy adecuado para **src/**

 analisis.ipynb

 Code

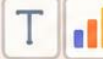
```
In [1]: import matplotlib.pyplot as plt
img = cargar_imagen("gato.jpg")

In [2]: plt.imshow(img)
plt.axis('off')

Out[2]: 
```

```
In [3]: # Explorar resultados
mostrar_mascaras(img)
```

 Notebook .ipynb

-  Documento dividido en celdas
-  Se pueden ejecutar celdas individualmente
-  El kernel mantiene un estado en memoria
-  Orientado a exploración y experimentación
-  Código, resultados, gráficas y texto conviven
-  Muy adecuado para **notebooks/**

Notebooks en VS Code.

Run All
Ejecutar todas las celdas de inicio a fin.

Reiniciar
Reiniciar el kernel (borra todas las variables activas del notebook).

Clear All Outputs
Borrar todas las salidas de las celdas de código.

Jupyter Variables
Muestra las variables activas en el notebook.

entorno-jupyter
Entorno que está ejecutando el notebook (nombre del entorno y versión de Python).

Celda de Markdown
Contiene texto en formato Markdown. Se puede incluir títulos, explicaciones, imágenes, etc.

Celda de código
Contiene instrucciones de Python que se pueden ejecutar. La celda se ejecuta con el botón ▶ (o con Run All si se quieren ejecutar todas).

Comentarios en código
Las líneas que comienzan con # son comentarios. Se usan para explicar el código y no se ejecutan.

Salida del código
Resultado que genera la ejecución de la celda de código. Puede ser texto, números, gráficas, tablas, etc.

Sección Jupyter Variables
Muestra las variables activas en el notebook, con su nombre, tipo, tamaño y valor.

Notebook in VS Code
Los archivos .jpynb permiten celdas con texto en formato Markdown.

```
[1] print("y también celdas para ejecutar código y desplegar su salida.")
y también celdas para ejecutar código y desplegar su salida.

[2] #Se pueden hacer comentarios en las celdas de código
    #Esto es un comentario

[3] # También se pueden observar las variables definidas en el notebook, así como su tipo y valor.
    a = 4
    print(a)
    print(type(a))
4
<class 'int'>
```

Name	Type	Size	Value
a	int		4

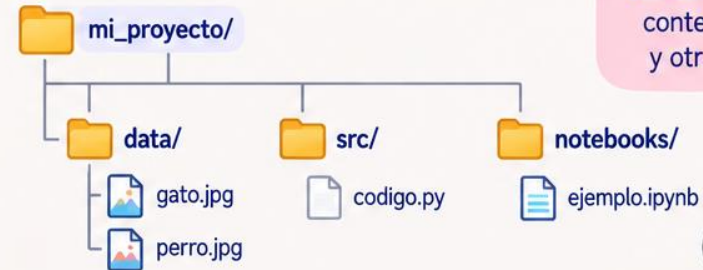
¿Cómo puede mi código ver los archivos de mi proyecto?

¿Qué son las rutas?

Una ruta (path) es la dirección que indica dónde se encuentra un archivo o carpeta dentro del sistema de archivos.

Una ruta me dice cómo llegar a un archivo o carpeta.

Ejemplo de estructura de carpetas



Las carpetas pueden contener archivos y otras carpetas.

Ruta absoluta

Indica la ubicación completa del archivo, desde la raíz del sistema de archivos. Siempre apunta al mismo lugar, sin importar en qué carpeta te encuentres.



Ejemplo en Windows

```
C:\usuarios\mi_proyecto\data\gato.jpg
```



Ejemplo en Linux

```
/home/usuario/mi_proyecto/data/gato.jpg
```

Ruta relativa

Indica la ubicación del archivo con respecto a tu carpeta actual (directorio de trabajo).

Si tu carpeta actual es **mi_proyecto**:

```
data/gato.jpg
```

Ruta al archivo dentro de la carpeta actual.

Si estás dentro de la carpeta **src**:

```
../data/gato.jpg
```

Ruta al archivo desde **src**, subiendo un nivel (..).

Directorios especiales: . y ..

Los directorios **.** y **..** se usan en rutas relativas para referirse a la carpeta actual y a la carpeta anterior.

Estoy en la carpeta **src**. Veamos qué significan.



Carpeta actual:
mi_proyecto/src



. Carpeta actual

El punto (.) se refiere a la carpeta en la que te encuentras.

```
./codigo.py
```

Es equivalente a:
codigo.py

.. Carpeta anterior

El doble punto (..) se refiere a la carpeta inmediata anterior (un nivel arriba).

```
../data/gato.jpg
```

Desde **src**, sube un nivel a **mi_proyecto** y entra a la carpeta **data**.

¿Qué pasa con diferentes sistemas operativos?

¡Te comparto mi código para cargar la imagen!

```
código.py  
  
data_dir = "data"  
imagen = "gato.jpg"  
ruta = data_dir + "\\" + imagen  
  
print(ruta)  
img = cv2.imread(ruta)
```

¡Perfecto!
Lo ejecuto en mi equipo...



En Windows

La ruta usa barras invertidas (\).

```
ruta = data_dir + "\\" + imagen  
data\gato.jpg
```



mi_proyecto

← → > data > gato.jpg



Archivo encontrado

La ruta coincide con la sintaxis esperada en Windows.



En Linux

La ruta usa barras normales (/).

```
ruta = data_dir + "\\" + imagen  
data/gato.jpg
```



mi_proyecto

← / > data > gato.jpg



No se encuentra el archivo

El separador fue escrito manualmente para otro sistema operativo.

Para evitar estos problemas, es mejor usar herramientas que se adapten a cada sistema operativo, por ejemplo pathlib:

```
from pathlib import Path  
ruta = Path("data") / "gato.jpg"  
img = cv2.imread(str(ruta))
```



Path construye la ruta utilizando las reglas del sistema operativo, sin tener que escribir manualmente el separador.